esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Advanced Learning for text and Graph Data



Chapitre 1:

Introduction au NLP : du traitement de texte à la construction de graphes

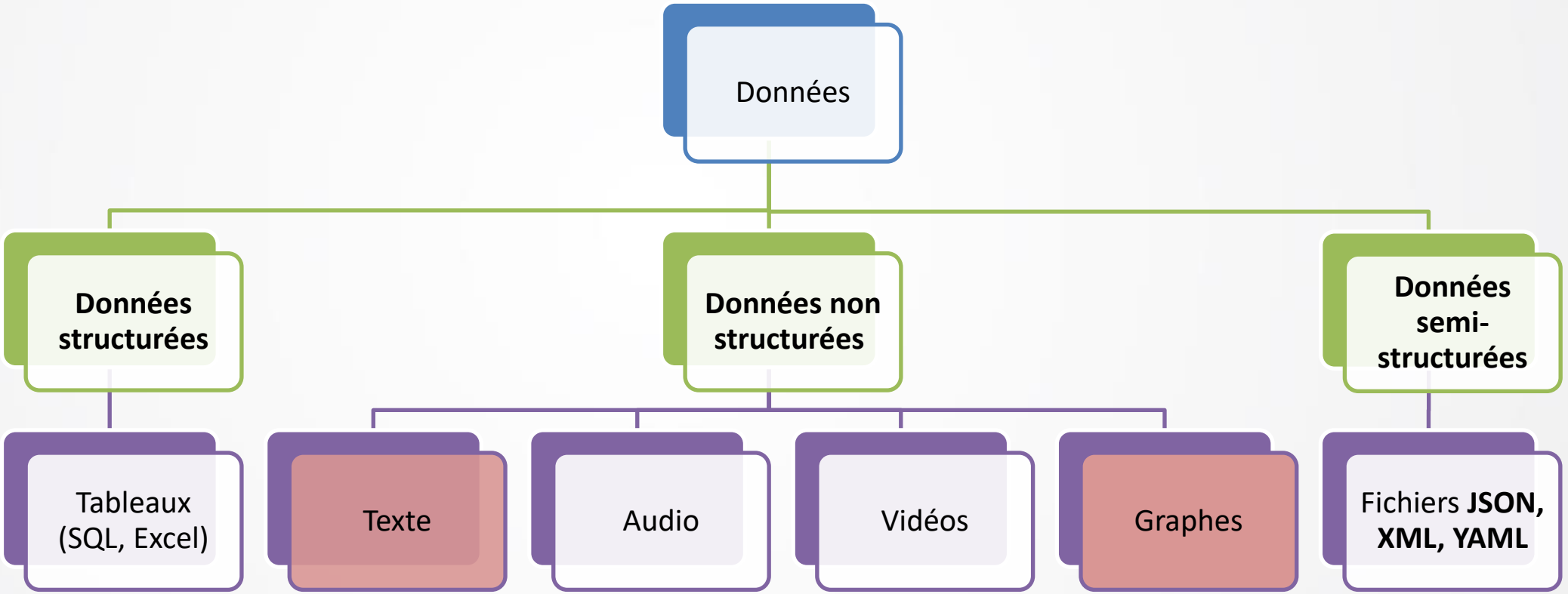
Unit: IL

United Nations Educational, Scientific and Cultural Organization
UNESCO Chair "Project-based learning" ESPRIT School of engineering, Tunisia
EUR-ACE® Délivrée par la Commission des Titres d'Ingénieur
CONFÉRENCE DES GRANDES ÉCOLES
cdio CONCEIVE DESIGN IMPLEMENT OPERATE

Plan

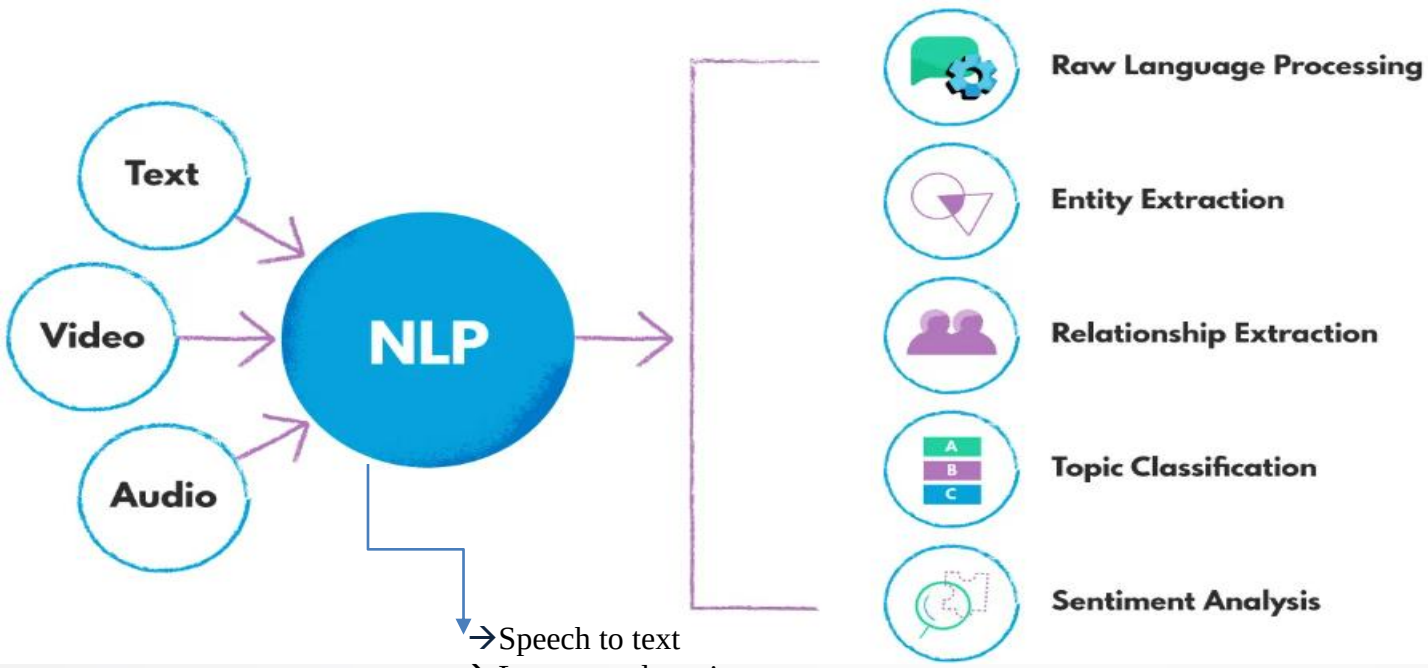
1. Introduction NLP: Qu’est-ce que le NLP ?
2. Notions de base en NLU (Natural Language Understanding)
3. Notions de base en NLG (Natural Language Generation)
4. Typologie des données Textuelles
5. Pré-traitement des données textuelles
6. Mesure de similarité textuelle
7. Introduction aux graphes dans le NLP
8. Typologie des données Graphes et hybride

Introduction_Advanced Learning for Text and Graph

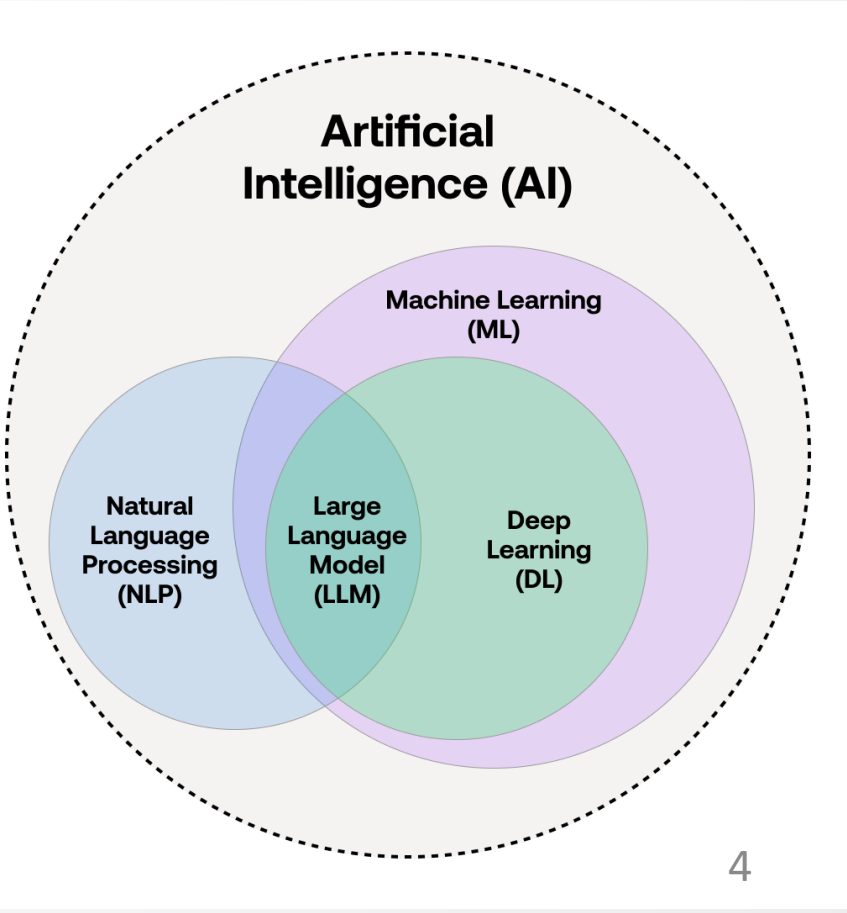


Qu’est-ce que le NLP ?

Traitement du Langage Naturel (ou **NLP – *Natural Language Processing***)
est un domaine de IA qui permet aux machines de **comprendre, interpréter, manipuler et générer** du langage humain (texte).



→Speech to text
→ Language detection
→Optical character recognition (**OCR)**



Défis du NLP pour le langage humain

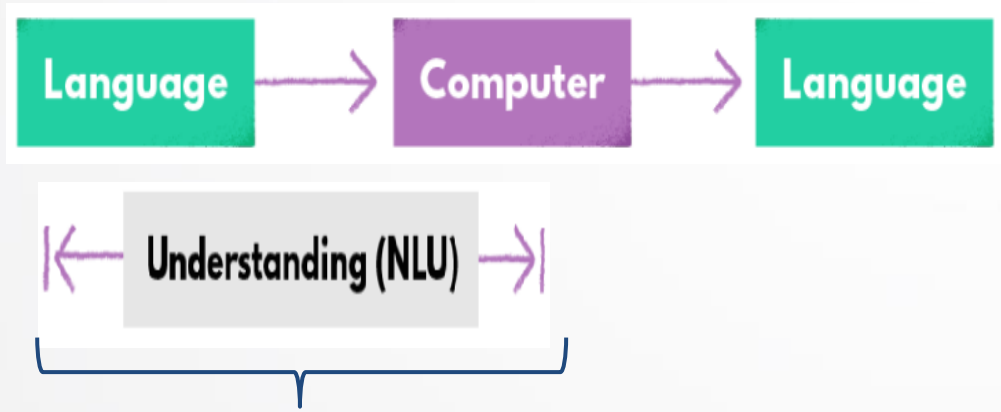
- Ambiguïté lexicale
- Polysémie et synonymie
- Langage informel et bruité
- Contexte et pragmatique
- Multilinguisme et variations linguistiques

Composantes principales du NLP : NLU vs NLG

1. Natural Language Understanding (NLU)

Permet à une machine de **comprendre le sens d'un texte ou d'une phrase**, en identifiant les entités, les intentions, les relations, ou encore la structure grammaticale:

→classification , parsing (analyse syntaxique) et extraction d'entité

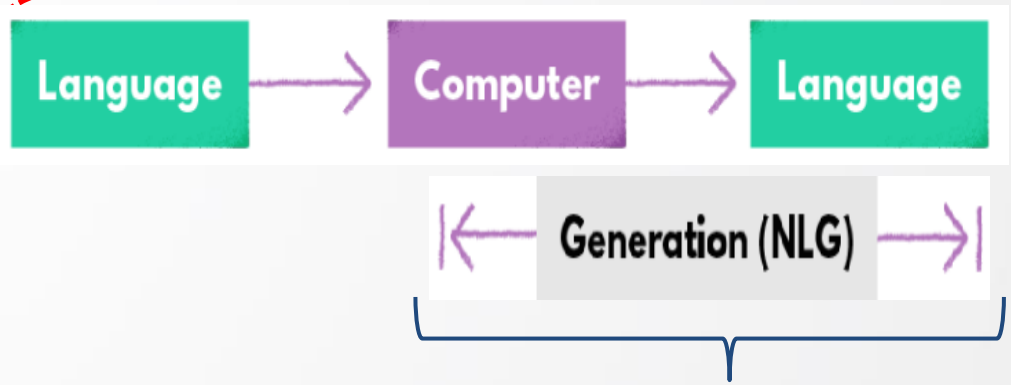


(1) La phase Linguistique : Data preprocessing (NLU)

2. Natural Language Generation (NLG)

Vise à **produire du texte** compréhensible et cohérent à partir de données ou d'une représentation sémantique:

→ Résumé, dialogue, rapport automatique

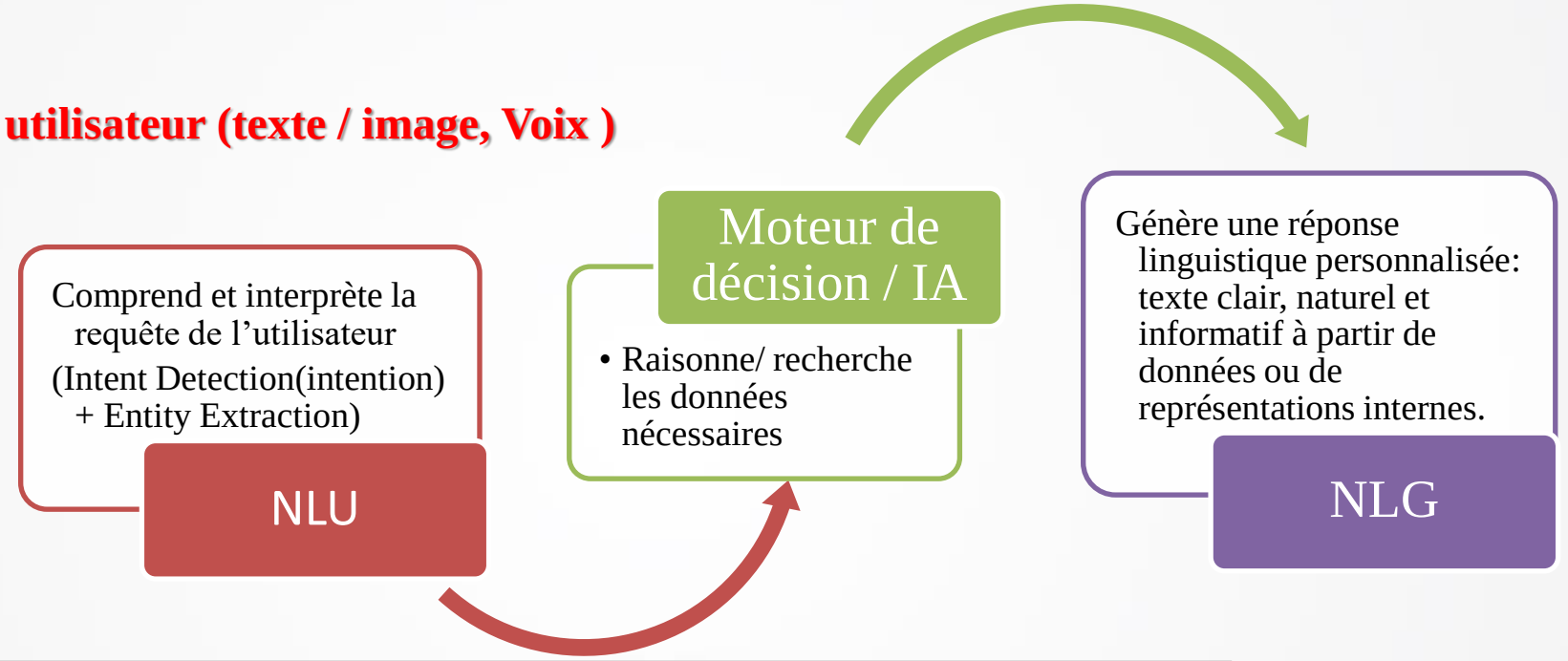


(2) La phase de l'apprentissage. automatique (NLG)

Les phases du NLP

Lien entre NLU et NLG dans les systèmes intelligents

→ **Input utilisateur (texte / image, Voix)**



→ **Output (texte ou voix via TTS)**

Exemple : Chatbot de réservation

Utilisateur : *"Je veux réserver un billet pour Paris demain "*

👉 **NLU identifie :**

- Intention : Réserver un billet
- Entité1: Lieu =Paris
- Entité2: Date = demain

Input JSON: { "lieu": "Paris", "date": "22 juillet", "confirmation": "OK" }

👉 **NLG produit :** *"Votre billet pour Paris a été réservé pour le 22 juillet. Merci !"*

Lien entre NLU et NLG dans les systèmes intelligents

Le système doit :

1. Comprendre la question (intentions, mots interrogatifs, focus):
"elle" fait référence à la tour Eiffel (coréférence)

2. Analyser le contexte (relations syntaxiques et sémantiques):
identifie le verbe "inaugurée"

3. Localiser l'information pertinente : la valeur temporelle associée

4. Extraire et générer une reponse adaptée

Exemple:

Contexte : "La tour Eiffel a été inaugurée en 1889.“

Question : "Quand a-t-elle été inaugurée ?”

Réponse attendue : "en 1889“

Notions de base en NLU (Natural Language Understanding)

- **Composantes fondamentales du NLU:**

Composante	Description
Segmentation	Découpage du texte en unités : phrases, tokens, sous-tokens (WordPiece, Byte-Pair Encoding)
Morphosyntaxe	Analyse morphologique et syntaxique (structure de la phrase, arbre de dépendance)
Sémantique lexicale	Interprétation du sens des mots en contexte (désambiguïsation lexicale)
Sémantique compositionnelle	Construction du sens global à partir de la structure syntaxique
Pragmatique	Interprétation du sens en fonction du contexte, des implicites, des actes de langage
Résolution de coréférence	Liaison des expressions référentielles aux entités qu'elles désignent

Notions de base en NLU (Natural Language Understanding)

- **Modélisation des connaissances:**

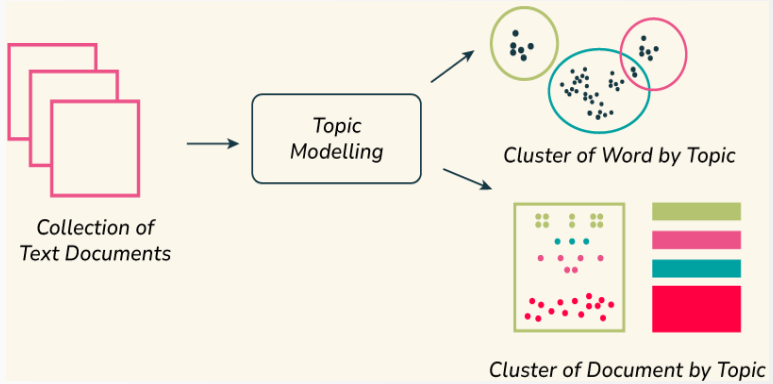
Le NLU requiert des modèles capables de représenter le sens des phrases:

Approche symbolique (Knowledge-based)	Approche apprentissage profond (Data-driven)
• **Ontologies** (WordNet, DBpedia, FrameNet) • **Règles logiques** : si/alors, logique de description • systèmes experts, moteurs d'inference	• **Word embeddings** : Word2Vec, GloVe • **Contextual embeddings** BERT, RoBERTa, GPT • **Fine-tuning** sur tâches spécifiques (NER, QA, NLI...)

Tâches avancées en NLU

1. Topic modelling

Une technique non supervisée de traitement du langage naturel qui permet d'**identifier automatiquement les thèmes (ou sujets latents)** présents dans un corpus de documents.



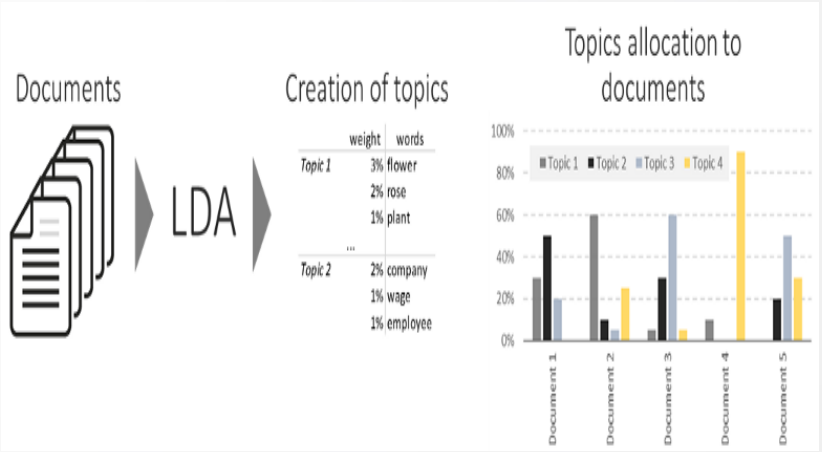
→ Un **document** est une **distribution de probabilités sur les topics**.
→ Un **topic** est une **distribution de probabilités sur les mots**.

Topic modelling avec LDA

Input: collection de documents
Technique : Latent Dirichlet Allocation (LDA)

- Prétraiter le texte: Nettoyage, tokenisation, Lemmatisation
- Construire vocabulaire et vectorisation : Bag-of-Words ou TF-IDF
- Appliquer le modèle LDA: nombre de topics K

Output: Une liste de thèmes qui définit les mots caractéristiques de chaque thème ainsi que les proportions d'affectation de chaque thème dans un document.



Exemple:

Topic 1 (Sport) → {football: 0.2, match: 0.15, joueur: 0.12, but: 0.08...}
Topic 2 (Cuisine) → {recette: 0.18, four: 0.14, gâteau: 0.13, sucre: 0.12...}

Tâches avancées en NLU

2. Reconnaissance d'Entités Nommées (Named Entity Recognition - NER)

→ Consiste à **identifier et classer automatiquement** dans un texte les **entités importantes** telles que : **Personnes** (ex : Macron), **Organisations** (ex : ONU, Google), **Lieux** (ex : Paris, Tunisie), **Dates** (ex : 14 juillet 1789), **Montants** (ex : 100 $)

Exemple: **Apple(ORG)** a annoncé le lancement de **l'iPhone 15 (product)** à **San Francisco (Location)**le **12 septembre (Date)**. "

In the 19th century, there was something called the "cult of domesticity" for many American women. This meant that most married women were expected to stay in the home and raise children. As in other countries, American wives were very much under the control of their husband, and had almost no rights. Women who were not married had only a few jobs open to them, such as working in clothing factories and serving as maids. By the 19th century, women such as Lucretia Mott and Elizabeth Cady Stanton thought that women should have more rights. In 1848, many of these women met and agreed to fight for more rights for women, including voting. Many of the women involved in the movement for women's rights were also involved in the movement to end slavery.

Tag colors:

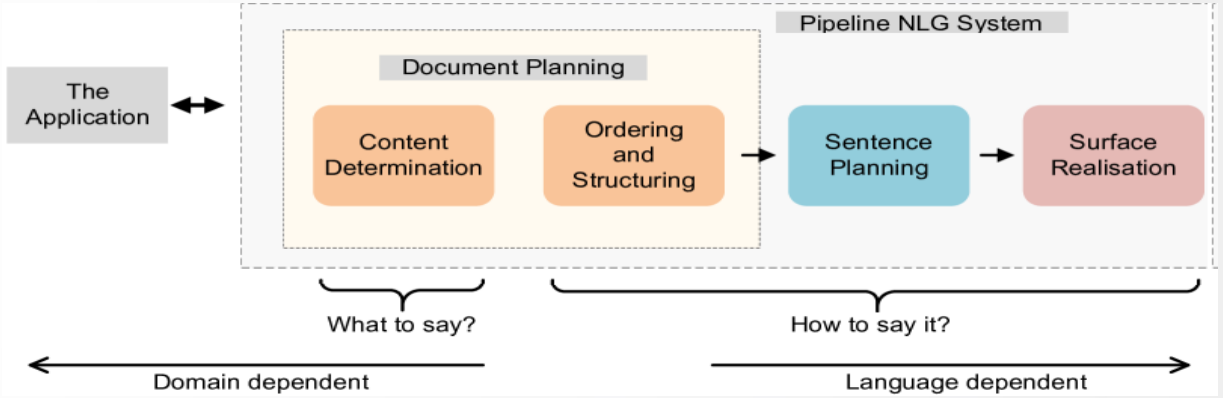
LOCATION PERSON TERM DATE CONDITION PROCESS PEOPLE

Notions de base en NLG (Natural Language Generation)

Le **défi du NLG est** de produire des textes **corrects**, **contextuellement appropriés**, **stylistiquement contrôlés** à partir de **divers sources** (tableaux, BDs, texte, graphes, signaux, images, historiques conversationnels etc.).

- **Le Pipeline NLG :**

L'architecture de NLG suit ces **étapes hiérarchiques** :



1. **Déterminer le contenu** : Quelles informations exprimer ?
2. **Structurer le discours :** Dans quel ordre les présenter ?
3. **planifier les phrases** :
   - Agrégation des idées
   - Choix des mots et expressions
   - Référence aux entités
4. **Génération syntaxique du texte** (phrases grammaticales)
5. **Post-processing** : correction orthographique, ponctuation, adaptation stylistique

- **Modèles & Architectures pour le NLG :**

- **BERT** (fine-tuned sur SQuAD, pour l'extraction de réponses)
- **T5**, **BART** → pour des réponses générées de manière plus naturelle
- **RAG** (Retrieval-Augmented Generation) → mélange recherche + génération
- **GPT-3/4, RoBERTa**, **DistilBERT**, **ALBERT** avec prompt + contexte

Notions de base en NLG (Natural Language Generation)

- **Méthodes de génération**

1. Approche Symbolique (Règle-based)

- Règles grammaticales.

2. Approche statistique / modèles probabilistes.

- Basée sur des **n-grammes**
- Génération simple mais peu créative.

3. Approche Neuronale

- *Encoder-Decoder (seq2seq) (resumé,traduction)*
- *Encoder-Decoder Transformers* (GPT, BART, T5…) Préentraînés en mode "Text-to-Text *(résumés, des questions, des paraphrases, des réponses ciblées)*
- *Causal Transformers (GPT-2/3/4, LLaMA, Mistral)* performants pour une génération libre, Supportent prompt engineering et fine-tuning adaptatif

Notions de base en NLG (Natural Language Generation)

- **Les métriques d'évaluation de la génération :**

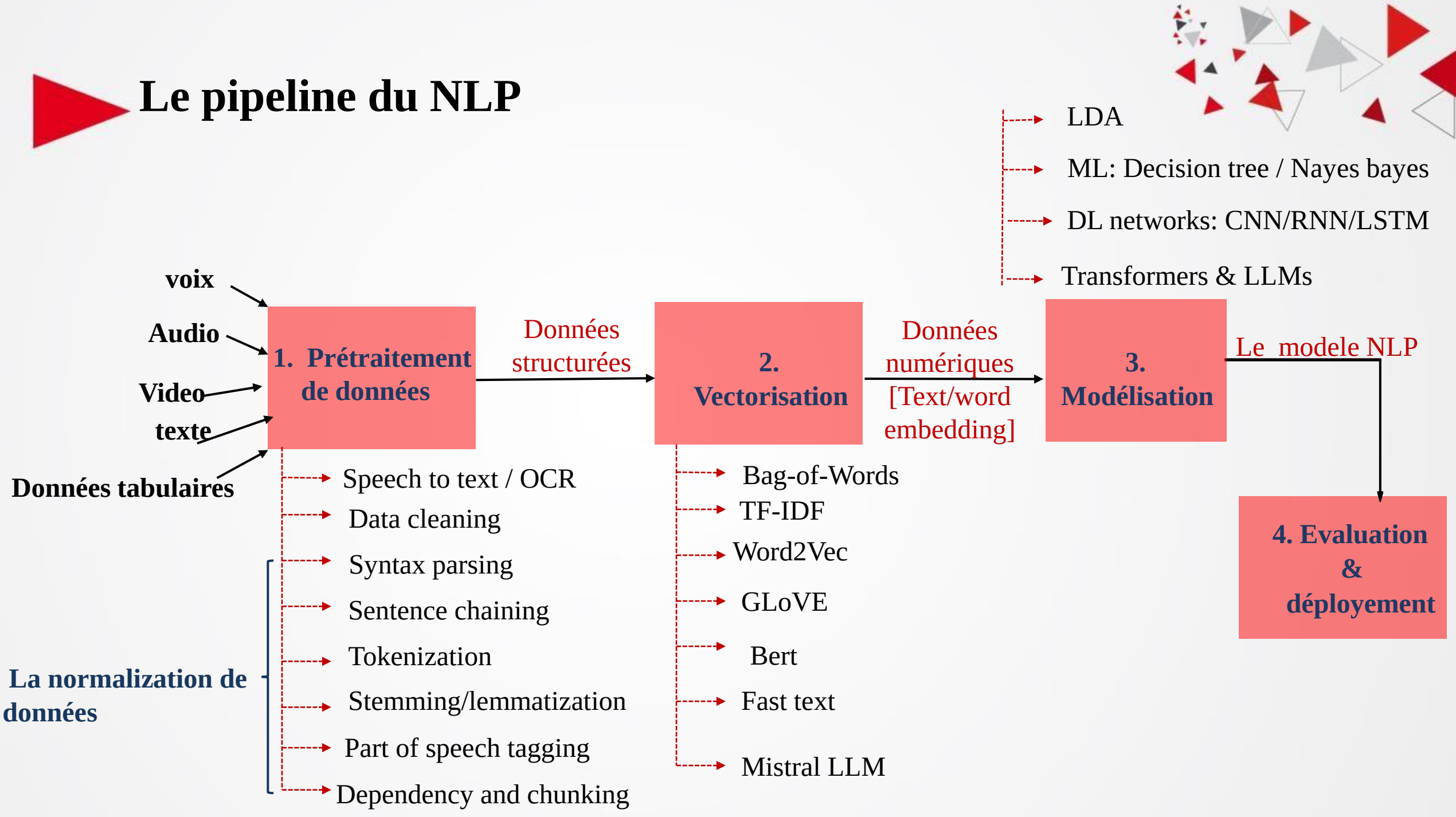
Évaluer la qualité d'un texte généré peut être automatique **ou** ~~humaine.~~

➔ **des métriques classiques (BLEU, ROUGE) ne capturent ni** le sens profond**, ni la** logique argumentative**.**

Métrique	Objectif
BERTScore	Similarité sémantique entre phrases (sens)
FactCC	Vérification de la factualité
QuestEval	Évalue via QA générées automatiquement
BLEU	Similarité avec une référence
ROUGE	Surtout pour le résumé

👉 NLG peut être **pilotée par des graphes de connaissances,** il génère du langage naturel à partir de structures sémantiques.

Le pipeline du NLP



Typologie des données Textuelles

Le **texte** est naturellement **linéaire**, mais il contient une structure **sémantique implicite**.
On le traite à différents niveaux de granularité :

Niveau	Description	Exemples de représentation
Caractère	Unité minimale (rarement utilisée seule)	One-hot, byte-level
Mot	Unité sémantique de base	Word2Vec, GloVe, fastText
Phrase / Énoncé	Structure syntaxique	RNN, LSTM, Transformers
Document / Corpus	Texte long et structuré	Doc2Vec, BERT CLS Token, SBERT

→ Les données textuelles peuvent être **non structurées**, **semi-structurées** ou **structurées:**

Type	Description	Exemples	Traitement
Non structurées	Texte brut sans balises ni structure logique	Articles, posts, emails, livres	Prétraitement + tokenisation
Semi-structurées	Texte avec une organisation partielle	JSON, XML, HTML	Parsing + extraction
Structurées	Texte avec structure tabulaire ou relationnelle	Champs de base de données, tableaux	Directement exploitable par les modèles

Prétraitement des données textuelles

Le prétraitement est une étape essentielle en NLP qui consiste à nettoyer et transformer le texte brut pour le rendre exploitable par un modèle.

"Alice a acheté des fleurs à la boutique."

1.Tokenization: Séparer la phrase en tokens
["Alice", "a", "acheté", "des", "fleurs", "à", "la", "boutique", "."]

2. Suppression des Stop Words :Retirer les mots très fréquents et peu informatifs
["Alice", "acheté", "fleurs", "boutique", "."]

3. Stemming :Réduire les mots à leur racine, souvent en coupant les terminaisons.
["Alice", "achet", "fleur", "boutiqu", "."]

4. **Lemmatisation :**Ramener les mots à leur forme canonique ou dictionnaire
["Alice", "acheter", "fleur", "boutique", "."]

5.Part-of-Speech Tagging (POS) Attribuer à chaque mot sa catégorie grammaticale.
Alice (Nom propre, NNP) / fleurs (Nom commun, NOM)

6.Named Entity Recognition (NER) :Identifier les entités importantes (personnes, lieux ..) "Alice" → Personne (PER)/"boutique" → Lieu (LOC)

7.Chunking: Former des groupes syntaxiques, par ex : syntagme nominal (SN), syntagme verbal (SV):
[SN: "Alice"][SV: "a acheté"]

+Vectorisation (BOW ,TFIDF ,Word Embeddings)

+Conversion en minuscules
+Suppression de la ponctuation …

La Vectorisation

La **vectorisation** consiste à **convertir des données textuelles brutes en vecteurs de nombres réels**, un format que les modèles d'apprentissage automatique peuvent traiter.

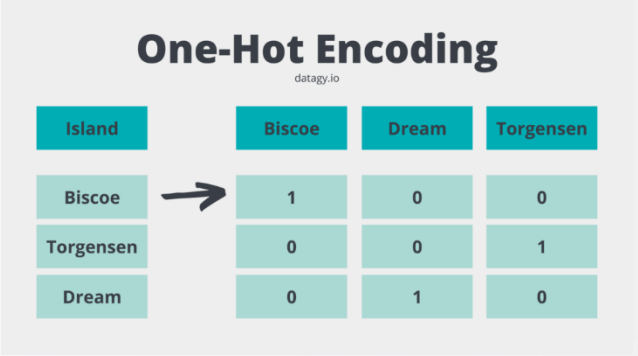
👉 Plusieurs types de vectorisation en NLP afin de représenter le texte sous forme numérique:

Les méthodes simples	
Sont faciles à comprendre et rapides mais ne capturent pas le sens ou le contexte.	(One-hot, BoW, TF-IDF)
Les embeddings statiques	
Représentent le sens des mots de façon dense mais un même mot a toujours la même représentation, quel que soit le contexte.	(Word2Vec, GloVe, FastText)
Les embeddings contextuels	
Produisent des vecteurs dynamiques qui varient selon la phrase, offrant une meilleure compréhension du sens selon le contexte, au prix d'une complexité et d'un coût calcul plus élevés.	(ELMo, BERT, GPT)

1. Approches traditionnelles de vectorisation

1.1 One-hot encoding

Chaque mot est représenté par un vecteur binaire .



Avantages : simple
Limites : pas de notion de sens ni de contexte, vecteurs très longs et creux.

1.2 Bag of Words (BoW): est la technique la plus simple. Elle se base sur trois étapes :



Tokenisation : le texte est découpé en mots (tokens).
Création du vocabulaire : on sélectionne uniquement les mots uniques parmi tous les tokens, puis on les trie alphabétiquement.
Création du vecteur : une matrice creuse est générée, où chaque ligne représente une phrase sous forme de vecteur. Les colonnes correspondent aux mots du vocabulaire, et les valeurs indiquent la fréquence d'apparition.
Ordre ignoré, seulement la présence/quantité.

Limites : pas de contexte, vecteurs dépendants du vocabulaire.

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

1. Approches traditionnelles de vectorisation

1.3 TF-IDF (Term Frequency – Inverse Document Frequency)

Pèse l'importance d'un mot dans un document .

Formule : TF-IDF(w) = TF(w, doc) × log(N / DF(w))

TF (Term Frequency) = **Fréquence du mot dans le document**

→ Combien de fois un mot apparaît dans *ce* document.

Il mesure **l'importance locale** dans un document.

$$TF = \frac{Frequency\ of\ word\ in\ a\ document}{Total\ number\ of\ words\ in\ that\ document}$$

IDF (Inverse Document Frequency) = **Importance du mot dans tout le corpus**

→ Plus un mot est rare dans l'ensemble des documents, plus son IDF est élevé.

$$IDF = \log\left(\frac{Total\ number\ of\ documents}{Documents\ containing\ word\ W}\right)$$

Il mesure **l'importance globale** dans tout le corpus.

(Un **corpus** est un **ensemble de documents ou de textes)**

Mots fréquents → moins importants (stopwords), mots rares → plus importants.

Soit un corpus de **3 documents** :

Doc1 : "le chat mange la souris"

Doc2 : "le chien mange l'os"

Doc3 : "le chat dort »

TF

"chat" apparaît **1 fois + 5 mots** au total

TF(chat,Doc1)=1/5=0.2

IDF

df(**chat)=2** + **NbrDoc=3**

IDFchat=log(3/2)≈0.176

3. TF-IDF

TFIDF(chat,Doc1)=TF(chat,Doc1)×IDF(chat)= 0.2×0.176≈0.0352

Un score faible → le mot n'est pas si informatif dans ce contexte.

2. Les embeddings statiques

2.1 Word2Vec:(Google, 2013) : est une technique de vectorisation avancée qui utilise un réseau de neurones simple pour apprendre des **représentations vectorielles (embeddings)** des mots .

Exemple :Le chien aboie fort

1. Modèle Skip-Gram:

À partir d'un **mot cible**, prédire les **mots du contexte**.
Convient mieux pour **petits corpus** et les mots **rares**.
Phrase tokenisée : ["Le", "chien", "aboie", "fort"]
Fenêtre (window size) = 1 (contexte gauche et droite)
Exemple :

Mot cible	Mots du contexte
chien	"Le", "aboie"
"aboie"	"chien", "fort"

Pour la paire ("chien", "Le"), le réseau apprend que "Le" apparaît souvent autour de "chien"

2. Modèle CBOW

À partir des **mots du contexte**, prédire le **mot central (cible)**
Plus rapide que Skip-Gram, mieux pour les **mots fréquents**.
Fenêtre = 1
Exemple :

Contexte	Mot cible
"Le", "aboie"	"chien"
"chien", "fort"	"aboie"

Le modèle apprend que si "Le" et "aboie" sont autour, alors le mot central probable est "chien".

2. Les embeddings statiques

2.2 GloVe (Global Vectors) :

Optimise des vecteurs pour prédire la fréquence avec laquelle les mots apparaissent ensemble

→ La **co-occurrence** entre deux mots signifie que ces mots **apparaissent ensemble dans un même contexte**

👉 Par exemple, nous avons les **2 phrases** suivantes :

1. « **Les données sont le nouveau pétrole.** »
2. « **Les données sont l'avenir.** »

À partir de ces deux phrases, nous pouvons construire la **matrice de cooccurrence** ci-dessous:

→ **P(J|I)=Xij/Xi**.

→ La probabilité que le mot **j** apparaisse dans le **contexte du mot i**.

- **Données** apparaît **2 fois** avec **est**:
  X(Données ,est)=2

- **Total des cooccurrences de est** :
  X(est)=2+1+1=4

- **P(Données | est) = 2 / 4**.

	Données	est	nouveau	pétrole	avenir
Données	0	2	0	0	0
est	2	0	1	0	1
avenir	0	1	0	0	0
nouveau	0	1	0	1	0
pétrole	0	0	1	0	0

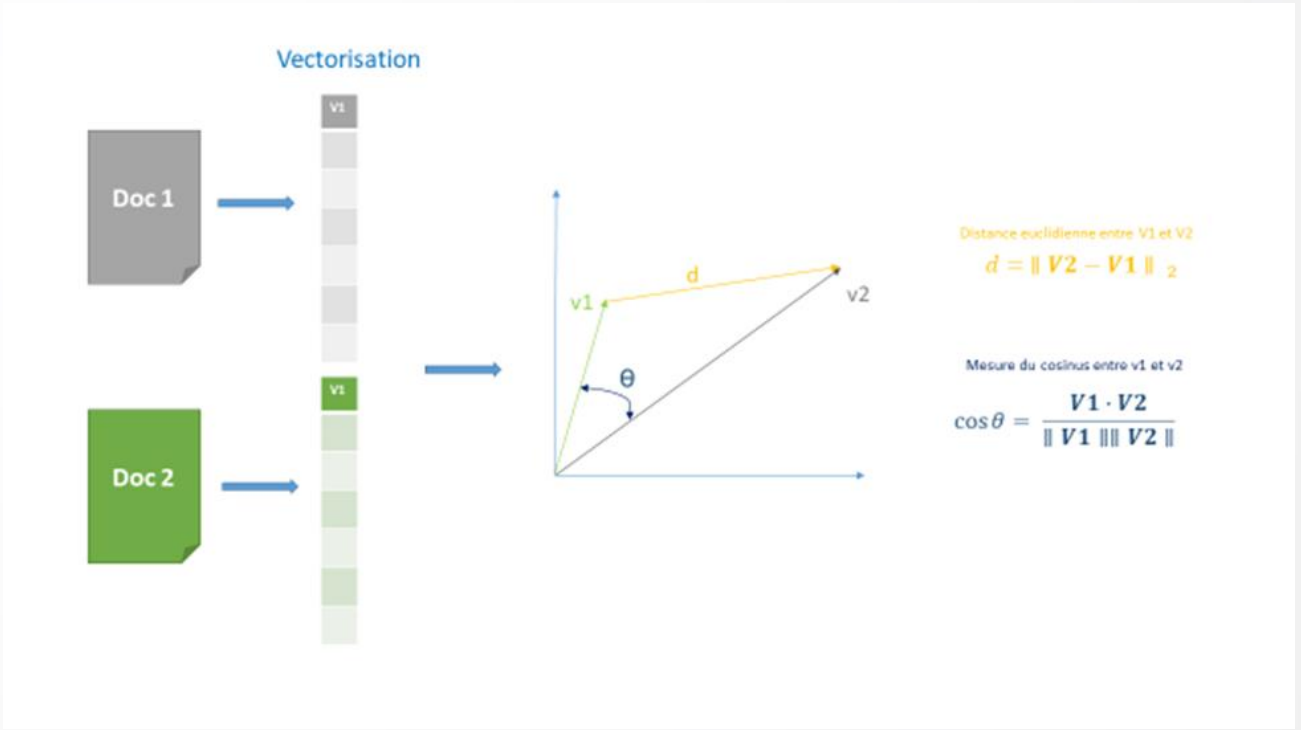
Mesure de similarité textuelle

La similarité textuelle en NLP permet d' évaluer la proximité sémantique entre documents ou phrases.

→ Grâce à la **vectorisation (embeddings)** , les textes sont représentés par des vecteurs numériques, facilitant le calcul automatique de leur proximité.

Exemples de mesures courantes :

- Similarité cosinus
- Distance de Levenshtein
- Distance euclidienne / Manhattan
- Similarité de Jaccard
- Mesures basées sur embeddings (Cosine similarity ,Dot product)



La distance de Levenshtein

La distance de Levenshtein est le nombre minimum d'opérations (insertion, suppression ou substitution d'un caractère) nécessaires pour transformer un mot en un autre.

Par exemple, la distance entre chat et chien est de 3 (remplacer 'a' par 'i', insérer 'n', remplacer 't' par 'e').
Une distance de 0 signifie que les mots sont identiques.

$$D(i,j) = \begin{cases} \max(i,j) & \text{si } \min(i,j) = 0 \\ \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + [a_i \neq b_j] \end{cases} & \text{sinon} \end{cases}$$

où $[a_i \neq b_j] = 0$ si les caractères sont égaux, sinon 1.

Similarité cosinus

La **formule de la cosine similarity** entre deux vecteurs $\vec{A}$ et $\vec{B}$ est :

$$\text{cosine_}similarity(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\| \vec{A} \| \quad \| \vec{B} \|}$$

où :

$\vec{A} \cdot \vec{B} = \sum_{i=1}^{n} A_i \times B_i$ est le **produit scalaire** des deux vecteurs.

$\| \vec{A} \| = \sqrt{\sum_{i=1}^{n} A_i^2}$ est la **norme (longueur) du vecteur** $\vec{A}$.

$\| \vec{B} \| = \sqrt{\sum_{i=1}^{n} B_i^2}$ est la norme du vecteur $\vec{B}$.

Exemple :

Phrase C : "J'aime le chocolat"
Phrase D : "Le chocolat est délicieux"
Vocabulaire :["J'aime", "le", "chocolat", "est", "délicieux"]

Vocabulaire : ["aime", "chocolat", "délicieux"]
Vecteurs :

$$\vec{C} = [1, 1, 0] , \vec{D} = [0, 1, 1]$$

Calcul de la cosine similarity :

Produit scalaire : $\vec{C} \cdot \vec{D} = 1 * 0 + 1 * 1 + 0 * 1 = 1$

Normes : $\| \vec{C} \| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2}$

$\| \vec{D} \| = \sqrt{0^2 + 1^2 + 1^2} = \sqrt{2}$

Cosine similarity : $1/(\sqrt{2} * \sqrt{2}) = 1/2 = 0.5$

La similarité cosinus entre "J'aime le chocolat" et "Le chocolat est délicieux" est **0.5**, ce qui montre qu'elles sont modérément proches en termes de sens, en se basant uniquement sur les mots significatifs : "aime", "chocolat", "délicieux".

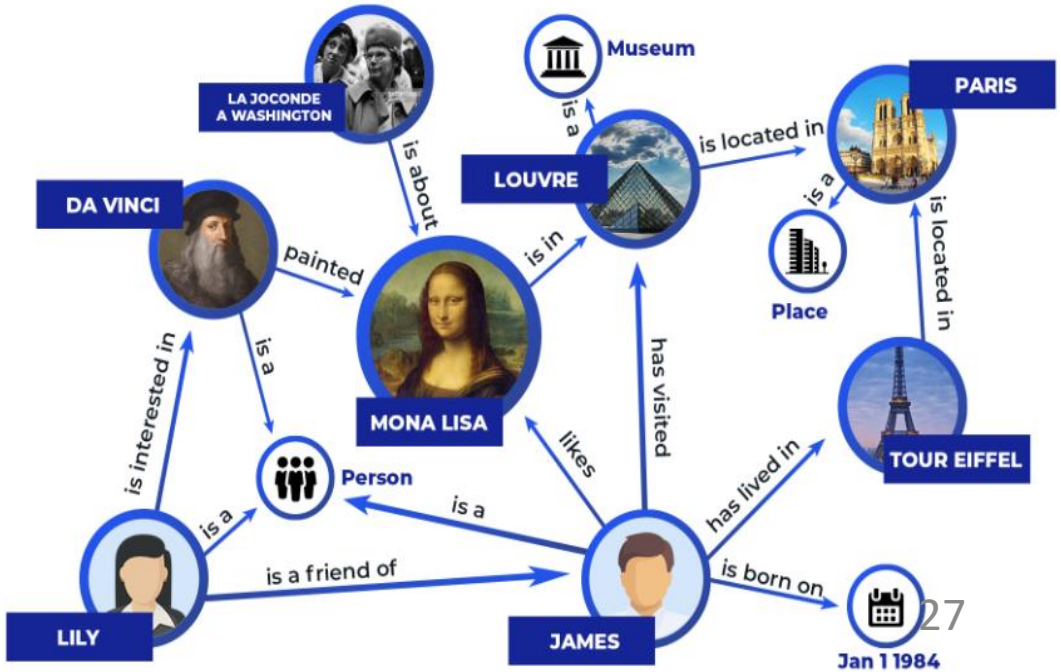
Qu’est-ce qu’un graphe ?

Un graphe est une structure mathématique constituée d’un ensemble de nœuds (ou des sommets) et un ensemble d’arêtes (ou des liens) qui les relient.

Les nœuds peuvent représenter n’importe quel type d’entité, telles que des mots, des expressions, des phrases, des documents ou des concepts

les arêtes peuvent représenter n’importe quel type de relation, telle que la similitude, la dépendance, la cooccurrence ou l’inférence.

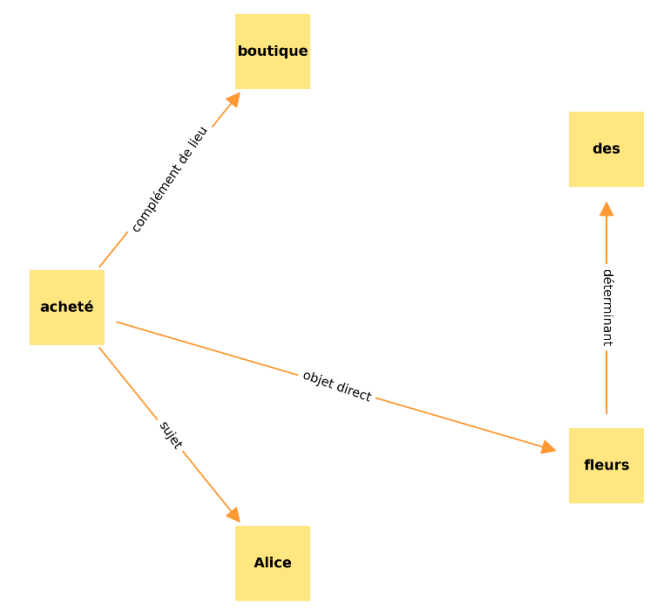
👉 Les graphiques peuvent capturer à la fois la **structure** et la **signification des données en langage naturel,** et permettre l’application de diverses opérations et algorithmes, tels que la traversée, la recherche, le clustering, le classement ou l’intégration.



Types de graphes en NLP

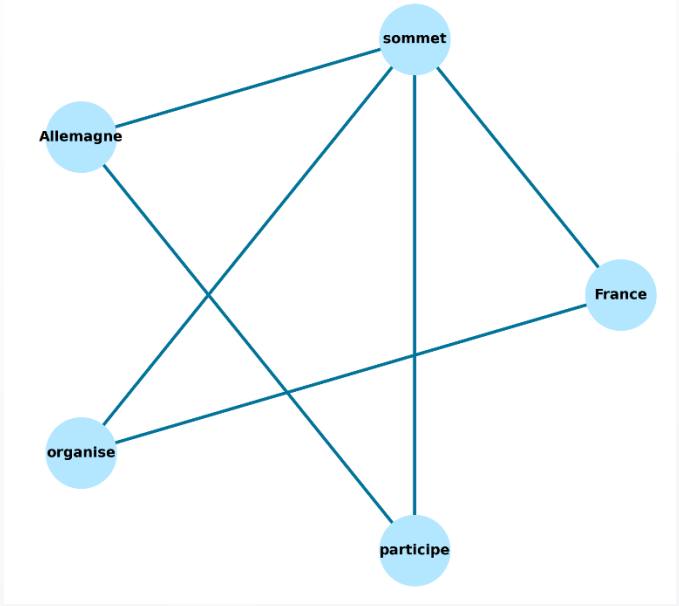
Graphe de dépendances Syntaxiques

Un graphe qui montre la **structure grammaticale** d'une phrase (qui est sujet, verbe, complément...).
Type : **Orienté**
Nœuds : les mots de la phrase.
Arêtes : les relations grammaticales (sujet, objet, complément…).
But : Comprendre "qui fait quoi »
"Alice a acheté des fleurs à la boutique. »



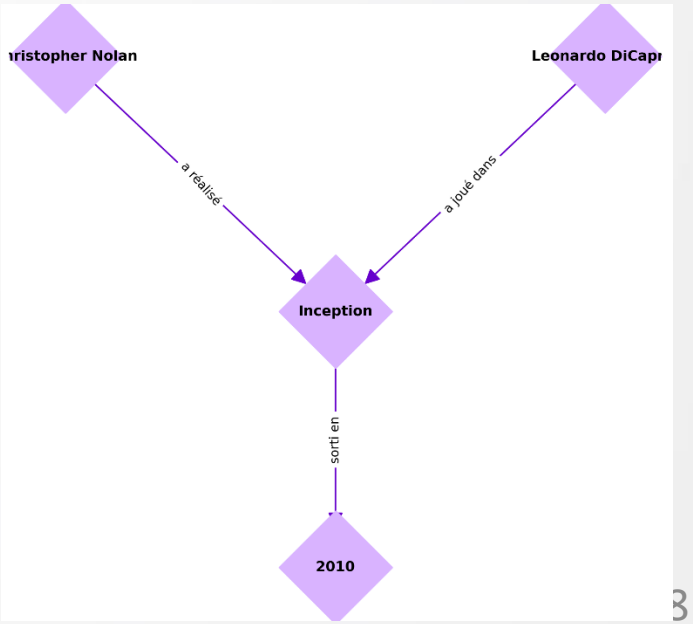
Graphe de similarite

Un graphe qui relie les mots qui **apparaissent ensemble dans un texte**, souvent dans la même phrase ou paragraphe
Type : **Non orienté**
Nœuds : mots.
Arêtes : fréquence de co-occurrence
But : Extraire des **motifs** ou des **relations contextuelles**.
"La France organise un sommet.
L'Allemagne participe au sommet."



Graphe de connaissances

Un graphe qui représente un **fait** sous forme de **relation entre deux entités**.
Type : **Orienté**
Nœuds : entités (personnes, lieux, objets...).
Arêtes : relations sémantiques (auteur de, situé à, possède…).
But : Modéliser le savoir réel ou les faits.
Chaque lien = **triplet (Sujet, Relation, Objet)**



Typologie des Données Graphes: structure explicite des relations

Un **graphe** est un ensemble de **nœuds (entités)** et **d'arêtes (relations)**. Cette structure permet de **modéliser les relations** complexes entre entités, souvent implicites dans un texte : **nœuds = objets / concepts** + **arêtes = relations ou interactions**

- **Taxonomie des graphes**

Type de graphe	Caractéristique	Exemple
Homogène	Un seul type de nœud/arête	Graphe social
Hétérogène	Multiples types (personne, entreprise...)	Knowledge Graph
Pondéré / dynamique	Poids ou évolution temporelle	Graphe de similarité

- **Représentations pour l'apprentissage:** description sémantique des nœuds via des vecteurs d'attributs (embeddings).

Représentation	Détails
Matrice d'adjacence	Représente la connectivité Matrix A(i,j) = 1 si lien
Feature matrix (X)	Attributs par nœud: TF-IDF, embeddings
Triplets (s, p, o)	Format RDF / KG
Subgraphs / motifs	Représentation locale pour classification ou clustering
Path embeddings	Chaînes logiques entre entités: Fichiers .edges

Typologie Mixte (Hybride): Texte + Graphe

Les systèmes modernes (notamment RAG, KG-QA, GNN-guided NLP) exploitent la **synergie entre texte et graphe** :

- **Texte vs graphe:**

Aspect	Texte	Graphe
Structure	Linéaire, séquentielle	Relationnelle, topologique
Représentation	Séquence, embedding	Adjacency matrix, edge list
Objectif	Compréhension sémantique	Raisonnement, propagation et inférence
Modèle	Transformers (BERT, T5)	GNN (GCN, GAT), KG Embeddings

- **Données Hybrides: texte enrichi par graphe, graphe extrait du texte**

L'avenir de la modélisation passe par la **fusion des deux typologies** :

- Extraction de graphes à partir de textes (NER, relation extraction)
- Alignement texte ↔ graphe (RAG, GNN-guided NLP)

Étapes de construction d’un graphe à partir d’un texte

1.Texte brut

Exemple : *"Paris est la capitale de la France."*

2.Prétraitement NLP(Tokenisation ,Lemmatisation/stemming ,Suppression des stopwords)

3.Définition des nœuds

1. Les nœuds peuvent être :
    1. des mots (graphe de cooccurrence)
    2. des entités nommées (graphe de connaissances)
    3. des syntagmes(un **groupe de mots** qui fonctionne comme une **unité grammaticale** dans une phrase. SN /SV)…

4.Définition des arêtes

1. Basées sur :
    1. la cooccurrence dans une fenêtre de mots
    2. les relations syntaxiques (dépendances)
    3. les relations sémantiques (ex. *capitale → pays*)

5.Construction du graphe

1. Nœuds reliés selon les règles choisies
2. Pondération des arêtes possible (ex. fréquence)

6.Représentation/visualisation

1. Via des bibliothèques comme **NetworkX**, **Gephi**, **Neo4j**, etc.

Références & Bibliographie

https://www.kdnuggets.com/top-picks-5-free-nlp-courses-recommend-2025
https://lena-voita.github.io/nlp_course.html#whats_inside_lectures

https://medium.com/@pundiramadhans/how-to-convert-any-text-into-a-graph-of-concepts-a841258d530c

https://blog.jcharistech.com/2025/03/10/building-a-text-knowledge-graph-in-python/

https://www.geeksforgeeks.org/nlp/build-a-knowledge-graph-in-nlp/

Radford et al. "Language Models are Few-Shot Learners", GPT-3 (2020)

Lewis et al. "BART: Denoising Sequence-to-Sequence Pre-training" (2020)

Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer", T5 (2020)

Yao et al. "KG-to-Text: Graph Neural Networks for Knowledge Graph to Text Generation" (2021)

https://medium.com/nerd-for-tech/word-embedding-text-analysis-nlp-part-3-glove-and-fasttext-da21d074237a

Questions?